

System Design

Red Bull Online Food-Delivery Data Platform

Data Engineering Case Study

This document covers architecture and data flow end to end, how the system scales as data volume grows or new markets are added, the data-quality and observability approach, and operational considerations for running this in production.

1. Architecture & Data Flow End to End

1.1 The pipeline

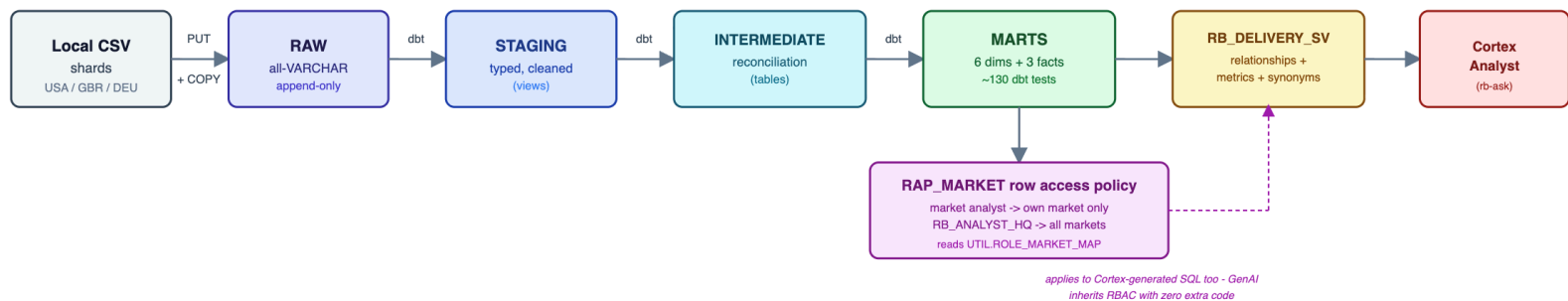
Stage	What it is	Materialization
Local CSV shards	Raw files from 3 delivery-platform feeds (outlet, portfolio, matching), 3 markets (USA/GBR/DEU), Q1 2024	files on disk
RAW	1:1 photocopy of the source files. All-VARCHAR, append-only, never edited.	tables
STAGING	Typed, cleaned, deduped — same grain as the source, no joins.	views
INTERMEDIATE	Grain-changing reconciliation: many messy rows → one clean row per real-world entity.	tables
MARTS	The star schema: 6 dimensions + 3 facts. What everything downstream actually queries.	tables
RB_DELIVERY_SV	A semantic view describing MARTS — relationships, pre-defined metrics, synonyms — for Cortex Analyst.	semantic view
Cortex Analyst	Natural-language interface over RB_DELIVERY_SV — translates English questions into SQL, executes under the caller's own Snowflake role, so RBAC composes automatically with zero extra code (rb-ask).	managed service (Cortex REST API)

1.2 Dataflow

- **RAW is a photocopy, never touched again.** All-VARCHAR, one COPY per file, nothing ever UPDATES a RAW row. If a downstream rule turns out wrong, RAW is the untouched source of truth to rebuild from — no re-download, no re-ingest.
- **STAGING cleans without changing grain** (views — cheap, always fresh, since the transforms are lightweight casts/renames with no aggregation). One row in, one row out, just typed and corrected.
- **INTERMEDIATE changes grain on purpose.** It's the one place multiple raw rows for the same real-world entity get resolved into one — int_outlet_unified (~1.7 listings → 1 outlet, mode-voted with conflict flags) and int_product_canonical (13.8M menu lines → 331 real drinks). Kept separate from STAGING and MARTS because it is a genuinely different kind of logic (reconciliation vs.

cleaning vs. shaping for consumption); mixing them would make each layer harder to reason about and test.

- **MARTS is the star schema** built for consumption by BI, the row access policy, and the GenAI semantic view alike. It is the only layer anyone outside dbt ever queries directly.
- **RBAC sits at the MARTS boundary, not earlier** — RAW/STAGING/INTERMEDIATE are invisible to analyst roles entirely (no schema grant), and the row policy then filters within MARTS by market. Two independent layers of defense.
- **The semantic view is a description, not a copy** — RB_DELIVERY_SV adds relationships, pre-defined metrics, and synonyms on top of the existing MARTS tables so Cortex Analyst can translate English into correct SQL. Because the generated SQL runs under the caller's own Snowflake role, RBAC composes into the GenAI layer automatically — `rb-ask --role RB_ANALYST_GBR "..."` is market-scoped with zero extra code.



1.3 Snowflake object inventory

Object	Count	Notes
Warehouses	3	RB_LOAD_WH (M, ingestion), RB_TRANSFORM_WH (S, dbt), RB_BI_WH (XS, analysts/GenAI) — sized to workload, all AUTO_SUSPEND=60
Schemas	5	RAW, STAGING, INTERMEDIATE, MARTS, UTIL
Roles	6	RB_LOADER, RB_TRANSFORMER, RB_ANALYST_{HQ, USA, GBR, DEU} — chained up to SYSADMIN for admin visibility
Mart tables	9	6 dims + 3 facts
dbt tests	~130	uniqueness, not-null, FK relationships, value ranges, flag-logic assertions

2. How the System Scales

2.1 Adding a market

A pure config change, no code:

- One line in ingest/feeds.yml under the relevant feed(s)
- One row in dbt/seeds/seed_market.csv
- One role (RB_ANALYST_<NEW>) + one row in UTIL.ROLE_MARKET_MAP

Everything downstream — the loader, the dbt models, the row access policy, the semantic view — already generalizes; none of them hardcode USA/GBR/DEU.

2.2 Adding a platform

Same idea — feeds.yml's market list already drives what gets loaded; a new platform just shows up as a new platform_name value and flows straight into dim_platform with no schema change, since platforms were never enumerated anywhere as a fixed list (the one test that pins the 6 known platform names is a guardrail to catch unexpected new ones, not a blocker — it fails loudly rather than silently, prompting a quick review).

2.3 Data volume growth

- **Warehouses resize independently** — RB_LOAD_WH (ingestion) and RB_TRANSFORM_WH (dbt) can each scale up without touching the other, because compute is already isolated by workload.
- **Full rebuilds stop being free.** At the current size, dbt build finishes in ~40 seconds; once the model is meaningfully larger, the standard next step is dbt's state comparison — store the manifest from the last successful main build and run dbt build --select state:modified+ in CI, rebuilding only what actually changed (detailed in cide_design.md).
- **Clustering keys** become worth adding on the largest fact (fct_menu_item, 13.8M rows today) once query patterns settle — e.g. clustering on market_key if most queries filter by market, which the row access policy already does for every analyst query.

2.4 Ingestion transport growth

The project deliberately separated transport (PUT from a laptop) from load (COPY INTO, defined once per feed in ingest/copy_templates/). Moving to production scale means swapping transport only:

Today: laptop — PUT → internal stage — COPY INTO → RAW

Production: scraper → S3 partition — Snowpipe (event-triggered) → RAW

The COPY body is unchanged — the same SQL that runs today under rb-load copy becomes the body of a CREATE PIPE ... AS <copy body> for Snowpipe auto-ingest. Everything downstream of RAW (dbt, RBAC, GenAI) is completely unaware of which transport landed the data.

2.5 GenAI layer growth

- **Verified Query Repository** grows organically from real usage — Cortex Analyst's own “Suggestions” panel surfaces repeated questions as candidates to verify, rather than requiring

someone to guess upfront which questions matter (see the propagation-timing finding in [genai/README.md](#)).

- **Cortex Search as a second tool**, alongside Cortex Analyst, would let an agent answer both “what does the data say” (structured, via the semantic view) and “what are the caveats” (unstructured, indexing docs/*.md) in one conversation
- **The eval gate scales with the model** — [genai/evaluate.py](#)'s gold set is 16 questions today; a production deployment needs 100+, sourced from real query logs rather than hand-written, and wired into CI so a schema change that breaks gold questions fails the PR instead of shipping quietly.

3. Data Quality Approach & Observability

3.1 Three independent checks on load correctness

Check	Source of truth	Catches
rb-load verify	independent Python re-parse of the local files	anything the first check could theoretically be fooled by — a second, unrelated opinion
dbt tests (~130)	the data itself, post-transformation	broken cleaning logic, broken joins, orphaned foreign keys, out-of-range values
genai/evaluate.py — 16-question gold set	hand-written reference SQL, run independently against Snowflake	execution-accuracy regressions, non-deterministic/drifted answers, invalid SQL, bad clarification handling

None of these is a single mechanism trusted blindly, they deliberately check different things, which is what let real issues surface immediately rather than silently.

3.2 The data-quality register

23 issues catalogued in [data_quality.md](#), each with its handling (fix / flag / defer) and the layer it's addressed in.

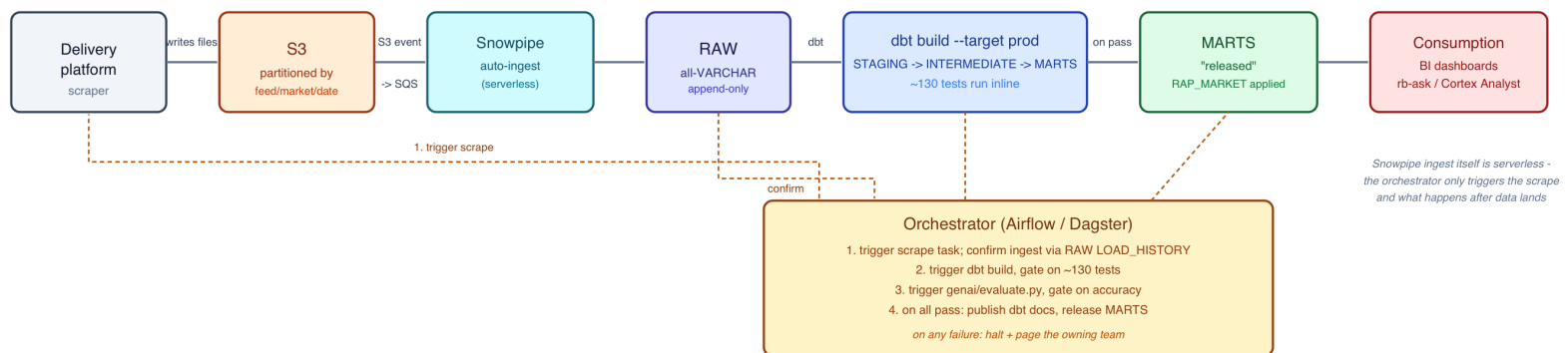
3.3 What would alert in a production deployment

- **A failing dbt test blocks the PR merge**, the same tests that run locally gate what reaches the real MARTS tables.
- **A RESOURCE_MONITOR** on each warehouse would catch runaway compute cost before it's a surprise on the bill.
- **A Snowflake ALERT** on `UTIL.LOAD_HISTORY` — e.g., "no successful load recorded for market X in the last 24 hours" — running on a schedule and firing a notification, so a silently-stopped feed is caught in hours, not discovered when someone finally asks why the numbers look stale.
- **The GenAI eval gate** ([genai/evaluate.py](#)'s 80% accuracy threshold) would catch a schema change that silently breaks Cortex Analyst's ability to answer known-good questions, run whenever the semantic view or a mart it depends on changes.

4. Operational Considerations — Production

4.1 Orchestration

In production, an orchestrator (Airflow/Dagster) replaces the manual make commands with one scheduled DAG: it triggers the scrape task directly (a step it fully owns), then merely confirms — via RAW's `LOAD_HISTORY` — that Snowpipe's separate, serverless `S3`→`RAW` ingestion has landed, since that hop isn't a task it controls. Once confirmed, it triggers `dbt build --target prod` (gated on the ~130 existing tests) and the GenAI eval harness (gated on the 16-question gold set) as release gates, publishing docs and marking MARTS "released" only if both pass — halting and paging otherwise.



4.2 Cost

- Every warehouse auto-suspends after 60 seconds idle — the right default for production too, refined per-warehouse only once real usage patterns justify keeping one warm.
- Cortex Analyst bills per message, not per token or per warehouse-second — a different cost shape from the rest of the platform, worth monitoring separately.
- `RESOURCE_MONITORS` per warehouse cap unexpected spend before it becomes a surprise.

4.3 Rollback

Because RAW is immutable and append-only, and STAGING/INTERMEDIATE/MARTS are fully rebuilt from it by dbt on every run, rolling back a bad model change is a git revert + dbt build, not a data-recovery exercise. This is a deliberate property of the layering, not an afterthought — the same reasoning that makes RAW “a photocopy, never touched again” valuable in the first place.